CapabilityBench: A Persistent-Life Benchmark for Embodied Agent Capabilities

Anonymous ACL submission

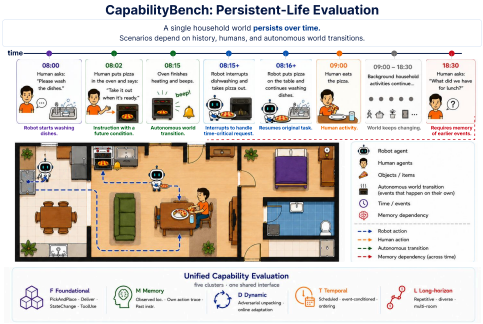

Figure 1: Visual abstract for CapabilityBench.

Abstract

Deploying LLM-driven embodied agents in real homes requires more than following one-shot instructions in a reset simulator: a useful household robot has to remember what happened earlier, react to a world that keeps changing around it, and act at the right moment in continuous time. Existing embodied benchmarks rarely stress these aspects, scoring a single episode at a time and reducing task pools to variants of pick-and-place. We introduce **LivingGrid**, a grid-based persistent household simulator in which time advances in fixed ticks, environment-driven transitions evolve the world on their own (a pizza keeps heating in the oven), and human agents driven by high-level intents follow daily routines that run before, during, and after any robot task. On top of LivingGrid we build **CapabilityBench**, a lifelong benchmark of 15 scenarios placed along a continuous timeline rather than reset around each task. The scenarios are procedurally instantiated from configuration files (sampling layouts, objects, humans, and timings) and cover four capability clusters relevant to real deployment: **Foundational** short-horizon manipulation and long-horizon decomposition, **Memory** across temporal gaps, **Dynamic** online adaptation to a changing world, and **Temporal** time- and event-conditioned action. We evaluate several language models in a shared tool-calling loop and compare them against published planning architectures (SayPlan, LookPlanGraph, AutoGPT+P).

1 Introduction

A useful household robot is not asked one isolated question at a time. It lives in the home: it has to remember what happened earlier, react to events it did not cause, and act at the right moment in a world whose time keeps flowing. Simulated or physical time advances whether or not the robot does anything, other agents act in the same world while the robot is working, and the robot is expected to remember and reason over its lived history (Wang et al., 2024). Questions like "where did I leave my glasses?" only make sense if the robot has been tracking the home across the gap between two interactions, and even strong language models struggle with information buried in long contexts (Levy et al., 2024; Modarressi et al., 2025).

Existing embodied benchmarks do not address such challenges. They are typically single-episode evaluations (Puig et al., 2018; Shridhar et al., 2020b; Li et al., 2024b; Wang et al., 2022): the simulator is reset, a one-shot instruction is issued, and the episode is scored in isolation. This reset-and-score pattern hides time that flows beyond a single task. Task pools are also often narrow (Choi et al., 2024; Su et al., 2024), reducing to pick-and-place variants, so the score reflects sentence-following rather than whether the assembled agent can operate in the environment.

We address this in two stages (Figure 1). **LivingGrid** is a grid-based persistent-life household simulator: scenes are multi-level scene graphs over a regular grid (rooms contain stationary assets, surfaces, and movable items with matryoshka-like containment), procedurally generated from short YAML configurations with a seed for reproducibility. Time advances in fixed ticks, and the world changes only through transitions: agent actions, autonomous dynamics (a heatable item becomes hot inside a powered microwave), and the routines of human residents driven by high-level intents. We

ship two reference houses (5-room/1-resident and 11-room/4-resident) for scaling experiments.

On top of LivingGrid we build **CapabilityBench**, a lifelong benchmark of **15 scenarios** placed along a continuous simulator timeline rather than reset around each task. Scenarios are organised into four capability clusters by the source of difficulty: **Foundational** (6 scenarios: instruction-grounded manipulation, delivery, state change, tool use, long-horizon decomposition), **Memory** (3, information moved to an earlier episode separated by simulated time), **Dynamic** (4, online re-grounding when the world changes under the agent), and **Temporal** (2, acting at a specified time or in response to a trigger). Each scenario fixes its milestone structure but resamples target objects, locations, humans, and instruction wording from the procedurally generated world.

We evaluate language-model agents along two axes. Varying the model inside a fixed ReAct-style tool-calling loop (Yao et al., 2023) gives a model-level capability profile. Holding the model fixed and varying the architectural wrapper—SayPlan (Rana et al., 2023) (scene-graph grounding), LookPlanGraph (Hu et al., 2023) (in-flight graph updates and correction), and AutoGPT+P (Liu et al., 2023) (PDDL planner as a tool)—isolates the contribution of the wrapper from that of the model.

Contributions. (i) **LivingGrid**, a procedurally generated persistent-life household simulator with time-driven transitions and intent-driven human agents behind a single action/observation interface. (ii) **CapabilityBench**, a 15-scenario lifelong benchmark organised into four capability clusters that single-episode benchmarks do not expose. (iii) A two-axis evaluation that separates the contribution of the language model from that of the planning wrapper across the four clusters. **TODO:** headline numbers.

2 Related work

Embodied simulators and task benchmarks. Household simulators such as VirtualHome, AI2-THOR, ProcTHOR, Habitat 2.0, and BEHAVIOR/BEHAVIOR-1K provide increasingly rich environments for object interaction, rearrangement, and everyday activities (Puig et al., 2018; Kolve et al., 2017; Deitke et al., 2022; Szot et al., 2021; Srivastava et al., 2021; Li et al., 2024a). Language-conditioned benchmarks such as ALFRED, ALFWorld, and TEACh connect such environments to natural-language instructions and dialogue (Shridhar et al., 2020a,b; Padmakumar et al., 2022). These benchmarks are essential for grounded instruction following, but they are typically organized around bounded episodes: a world is initialized, an instruction is issued, the agent acts (sometimes with a long horizon), and success is scored locally. CapabilityBench instead evaluates agents in a persistent household timeline that continues to evolve independently of the evaluated robot's actions.

Several recent benchmarks target individual pressures that CapabilityBench combines. PARTNR and Watch-And-Help study human-aware assistance and collaboration (Chang et al., 2024; Puig et al., 2021). FindingDory and 3DLLM-Mem study embodied memory over prior experience (Yadav et al., 2025; Hu et al., 2025). HAZARD, Robotouille, and ET-Plan-Bench stress dynamic, asynchronous, or spatial-temporal planning (Zhou et al., 2024; Gonzalez-Pumariega et al., 2025; Zhang et al., 2024). LongAct targets long-horizon household execution with dependency management and memory maintenance (Zhu et al., 2026). CapabilityBench differs in the unit of evaluation: not a single task, memory query, collaboration episode, or asynchronous plan, but a sequence of scenarios embedded in the ordinary life of the same continuing home.

Agent architectures. CapabilityBench is orthogonal to LLM planning methods. ReAct gives a minimal reason-act loop (Yao et al., 2022), while SayCan, Inner Monologue, SayPlan, AutoGPT+P, and LookPlanGraph add affordance grounding, execution feedback, scene graphs, symbolic planning, or graph updates around an LLM (Ahn et al., 2022; Huang et al., 2022; Rana et al., 2023; Birr et al., 2024; Onishchenko et al., 2025). These systems are candidate agents or baselines; CapabilityBench supplies the shared persistent-life environment, observation/action contract, scenarios, and milestone scores under which they can be compared.

3 LivingGrid environment

We introduce **LivingGrid**, a grid-based environment for simulating realistic indoor spaces. LivingGrid combines a semantic simulation of object placement over a multi-level scene-graph hierarchy, a system of agents whose behaviour can be specified through high-level intents, and an extensible

Benchmark	Foundational Atomic	Foundational Long-H	Memory Multi-ep.	Dynamic World	Dynamic Human-aware	Temporal Time/Event
VirtualHome (Puig et al., 2018)	✓	✓	×	×	×	×
BEHAVIOR-1K (Li et al., 2024a)	✓	✓	×	×	×	×
PARTNR (Chang et al., 2024)	✓	✓	×	×	✓	✓
FindingDory (Yadav et al., 2025)	✓	✓	×	×	×	×
3DLLM-Mem (Hu et al., 2025)	✓	✓	×	×	×	×
Watch-And-Help (Puig et al., 2021)	✓	✓	×	×	✓	×
Robotouille (Gonzalez-Pumariega et al., 2025)	✓	✓	×	✓	×	✓
LongAct (Zhu et al., 2026)	✓	✓	×	✓	×	✓
CapabilityBench	✓	✓	✓	✓	✓	✓

Table 1: Comparison with related embodied benchmarks. Atomic denotes grounded object or environment interaction. Long-H denotes long-horizon decomposition or sustained embodied task execution. Multi-ep. denotes multi-episode scenarios whose dependencies are separated by simulated time and intervening household activity inside the same continuing world. World denotes task-relevant state changes without the evaluated robot's direct action. Human-aware denotes other agents acting during or between tasks. Time/Event denotes scheduled, delayed, event-conditioned, or dependency-sensitive execution.

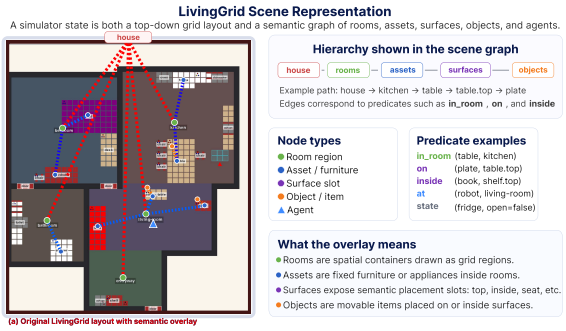

Figure 2: Scene representation of LivingGrid.

set of semantic transitions that let the world change through agent actions or through autonomous environment dynamics.

3.1 Semantic grid

LivingGrid uses a regular grid as a deliberately simplified model of indoor space, reducing room shape, object footprint, distance, reachability, and collision to discrete cells. Continuous control, manipulation, and perception are abstracted away so that evaluation targets high-level planning rather than low-level execution: the agent decides *what* to do and *in what order*. Over this grid we add a semantic layer. Every entity is described by three fields: a *type*, a set of *properties*, and a dictionary of *states*. The type is a single string such as `apple` or `fridge`. Properties are an open-ended list of strings that attach stable semantics to the entity at creation time, for example `red`, `cuttable`, or `half`. States are an open-ended key-value dictionary that can change during simulation, for example whether a fridge is open, whether a lamp is powered, or whether a plate is dirty.

To combine these spatial and semantic views, LivingGrid represents each scene as a scene graph whose edges encode placement relations such as *on* and *inside*. The main node classes are rooms, assets, items, surfaces, and agents. Rooms are physical regions of the grid. Assets are stationary objects such as furniture: they occupy grid cells and also carry the same semantic fields as other entities. Items are movable objects that the agent can pick up, carry, and place, changing both their semantic parent in the graph and, when placed on the floor, their grid position. Agents are embodied actors with poses in the same grid and inventories in the same graph. Surfaces are semantic placement spaces attached to rooms, assets, or items: the top of a table, the inside of a drawer, a shelf in a cabinet, or the inside of a lunchbox. Because items can themselves own surfaces, the graph supports matryoshka-like containment: an apple can be inside a lunchbox, the lunchbox can be on a counter, and the counter can stand in the kitchen.

3.2 Transitions

The state of LivingGrid changes only through transitions. A transition is a state-dependent rule that checks whether it is applicable, mutates the environment or the states of entities, and records the resulting event. Some transitions are explicit effects of agent actions, such as moving an item from a surface into an inventory or opening a closed container. Others are autonomous dynamics: the `advance_time` transition moves simulated time forward by ten seconds each tick, and a heating transition can mark a `heatable` item as hot while it is inside a powered microwave. This makes dynamics extensible at the semantic level: adding a new verb, process, or object-state update means adding a new transition rather than changing the

simulator loop. Within each tick, agent transitions are applied first, followed by world transitions.

3.3 Agents

Agents can execute the following 15 primitive actions: `move_forward`, `turn_left`, `turn_right`, `pickup`, `place`, `open`, `close`, `turn_on`, `turn_off`, `give`, `cut`, `pour`, `wash`, `wait`, and `say`. To simplify semantic control, agents can also carry externally assigned intents. An intent is a high-level command such as `go_to(kitchen)` or `wash(plate)`. Each intent is served by an extensible state machine handler that uses simulator-side ground-truth state to emit the next primitive action needed under the current world configuration.

To simulate communication, which is central to instruction following, `say` temporarily writes a message with its sender, text, and time into the environment state, so instructions and updates are delivered inside the simulated world rather than through an external dialogue channel.

In the current interaction model, an agent acts through a 3×2 window in front of it. Transitions use this window as part of their applicability condition: a pickup transition, for example, works only if the agent chooses to pick up an item and that item is reachable through the scene graph from an entity in the interaction window.

3.4 Procedural generation

To support diverse indoor layouts, LivingGrid generates scenes from YAML configuration files rather than fixed maps; representative fragments are shown in Appendix C. A configuration defines the asset/item/surface catalogue, room templates, and a house template with room proportions and connectivity. The generator samples room sizes, places rooms to satisfy the adjacency graph, and places furniture under footprint and approachability constraints. Floor furniture uses a wall-walk procedure: the generator walks along the interior wall path, tries candidate assets against oriented footprint, free-side, and front-clearance constraints, and reserves occupied and approach cells. It then populates surfaces from allowed-object lists and initializes agents. The result is a filled, configurable, traversable layout; a random seed fixes the sampled world for reproducibility.

4 CapabilityBench

To evaluate agents in a long-running embodied world, we introduce **CapabilityBench**, a benchmark built around LivingGrid. CapabilityBench places a robot in a text-based grid simulation of an inhabited home, with action and observation constraints intended to approximate the limits of a real household robot. Human agents follow daily routines over several simulated days, and benchmark scenarios are inserted into this continuing life as either single-episode interactions or multi-episode interactions whose parts may be separated by time gaps and by other intervening episodes. We organise tasks into a capability taxonomy, score robot behaviour through milestones, and aggregate the results into metrics that expose which capabilities are missing or fragile.

4.1 Life-long evaluation

CapabilityBench evaluates a robot in a continuing household simulation rather than in isolated reset-and-solve tasks. Each benchmark run contains a sequence of scenarios inserted into the ordinary activity of the home. Human agents continue to follow their routines between and during these scenarios, so the robot must operate in a world where task-relevant events are mixed with background life. While an episode is active, it may temporarily overwrite the intents of the participating human agents; outside that episode, control returns to their routine policy.

The basic evaluation unit is a *scenario*. A scenario is an ordered set of *episodes*, and an episode is a temporally local interaction with the robot: a request, an observation opportunity, a world change, or a phase of a longer task. Episodes may be separated by simulated time during which other routines or episodes of other scenarios occur. Scenario instances vary their natural-language instructions and sample target objects, locations, and human agents randomly, while respecting scenario-specific constraints and common-sense compatibility. This makes it possible to express tasks such as remembering where a human hid an object, returning an item to a location used in an earlier interaction, or reacting to a later instruction that refers back to a previous one.

Episodes are scored through weighted *milestones*. A milestone is a task condition that becomes latched once satisfied, such as picking the right object, opening the relevant container, delivering the object to the correct human, or completing the final placement. The scenario score is the weighted progress over its episodes. This gives partial credit for meaningful intermediate behaviour

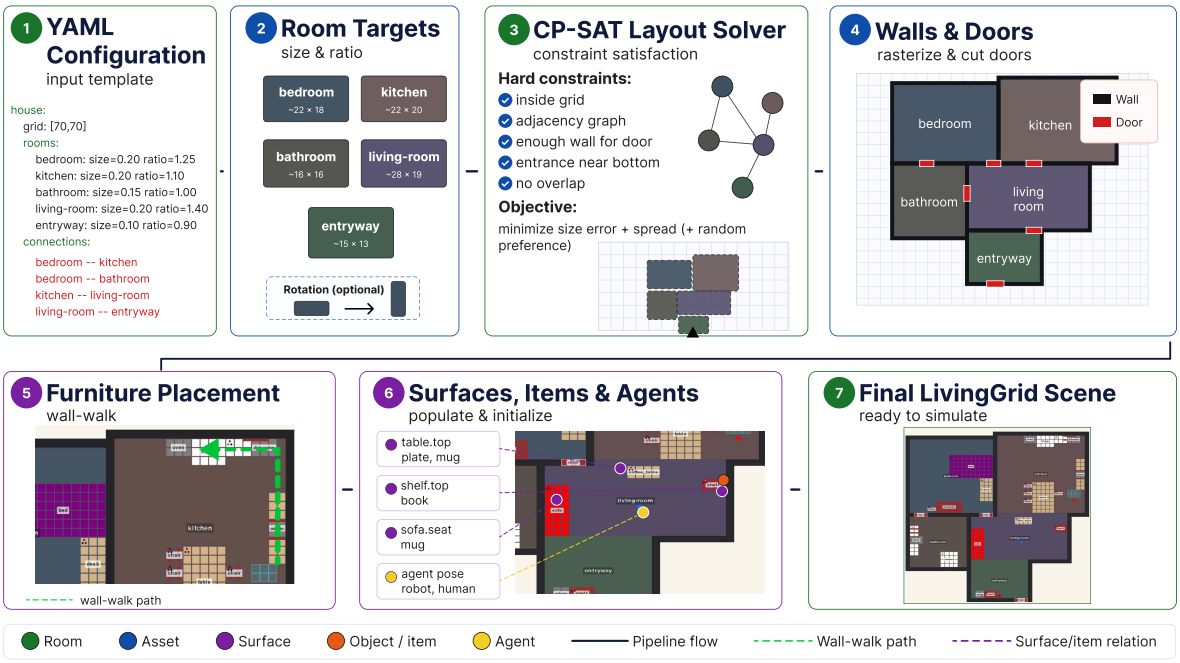

Figure 3: Generation pipeline of LivingGrid.

while still exposing which capability failed when the final task is not completed.

4.2 Capabilities

CapabilityBench is organized around a capability taxonomy rather than a flat list of household tasks. The taxonomy lets the same milestone-based scoring expose not only whether an agent completed a scenario, but also which class of competence is missing or fragile. We define four clusters by the source of difficulty: the required grounding and decomposition scale, the need to preserve information over time, changes in the world during execution, and temporal constraints on when to act.

Foundational scenarios test whether the agent can ground an instruction in visible entities, navigate to the relevant place, and execute the required manipulation or state-change steps. Short-horizon variants check basic grounding and single-step execution; long-horizon variants additionally stress decomposition, requiring the agent to break one instruction into many grounded subtasks, route through multiple rooms or object classes, and keep track of partial completion. **Memory** scenarios keep the physical task simple but move the crucial information into an earlier episode separated by simulated time: the agent must decide what to retain and later retrieve. **Dynamic** scenarios keep the goal active while the environment changes, so success depends on adapting online to new world states, human actions, instruction updates, or execution failures. **Temporal** scenarios differ from dynamic ones because the relevant condition is time or an event trigger rather than an unexpected spatial change; the agent must understand what to wait for, how long to wait, and how to preserve the pending task even when other interactions occur. The full capability table and detailed milestone definitions are given in Appendix D.

4.3 Agent contract

The agent contract is designed to approximate the information that a household robot could obtain from perception and navigation modules. At each decision point, the robot receives a structured JSON observation, and the environment accepts the robot's next action in JSON format. The observation contains the current time, the robot position, the known house layout with rooms and furniture, messages emitted in the current step, events produced by the simulator in the current step, textual feedback from the previous action, and a surface-level view of entities in the house. This view includes objects and surfaces that are exposed anywhere in the layout, while hidden contents are revealed only through open containment chains: opening a fridge can reveal its reachable contents, but objects inside a closed freezer nested in that fridge remain hidden. The full JSON schema is in Appendix A.

The robot action space follows the primitive actions described for LivingGrid agents in Section 3.3, but the robot does not receive LivingGrid intents because their handlers may rely on privileged simulator state. The benchmark contract also adds two restrictions. First, the robot cannot use speech as an action. Second, in addition to `wait`, the robot can call `sleep(duration)` to yield simulated time for a chosen interval; this sleep is interrupted when a message is produced in the environment.

4.4 Metrics

A single success number hides whether an agent failed immediately, made steady partial progress, or solved the task at an impractical cost. We therefore report four complementary metrics: success rate, per-scenario milestone score, executability, and token cost.

Success rate. For a set of launched episodes $\mathcal{E}$, $\mathrm{SR} = N_{\mathrm{full}}/|\mathcal{E}|$, where N_{full} is the number of episodes completed with all milestones closed. We report SR per scenario and averaged within each capability cluster.

Per-scenario score. For an episode e, S_e is the sum of weights of completed milestones, with milestone weights normalized to sum to one. For a scenario s, $S_s = \mathrm{mean}_{e \in \mathcal{E}_s} S_e$, where $\mathcal{E}_s$ is the set of episodes in the scenario.

Executability. For submitted actions $\mathcal{A}$, $\mathrm{Exec} = N_{\mathrm{accepted}}/|\mathcal{A}|$, where N_{accepted} is the number of actions applied by the simulator without rejection.

Token cost. For a scenario s, $\mathrm{Cost}_s = p_{\mathrm{in}} T_{\mathrm{in},s} + p_{\mathrm{out}} T_{\mathrm{out},s}$, where $T_{\mathrm{in},s}$ and $T_{\mathrm{out},s}$ are input and output tokens, and p_{in}, p_{out} are the published per-token prices for the model.

4.5 Protocol

For each evaluated agent, we run 50 full LivingGrid rollouts. A rollout spans several simulated days and contains the complete schedule of benchmark episodes across all scenarios, inserted into the continuing routine life of the house. Each rollout seed fixes the generated layout, object placement, human routines, scenario bindings, and episode schedule. The same 50 seeds are used for all agents, so comparisons are paired.

At each robot decision point, the agent receives the JSON observation defined in Section 4.3 and returns a JSON action. The runtime applies the robot action, applies the active episode intents for participating human agents or their routine intents otherwise, advances environment transitions, and then updates episode milestones. If the robot calls `sleep(duration)`, the runtime advances simulated time until the duration ends or a message is produced in the environment.

Metrics are computed at the episode and scenario level for each rollout, then averaged across the 50 seeds. Scenario-level results are additionally aggregated by capability cluster. We report averages across the 50 rollout seeds. All LLM calls use temperature 0.

5 Experiments

This section sets out the result tables that will be populated for the camera-ready version. All result cells are placeholders.

5.1 Baselines

We evaluate two common ways of extending an LLM agent under the same external contract. The first is ReAct-style control, where the language model loop is extended primarily through tools.

ReAct. A ReAct-style loop drives the robot one decision at a time. At each step, the loop renders the current observation as JSON, appends the action schema, and asks the language model to choose the next action. The baseline keeps a dialogue-like context memory over the last fifty simulator events.

Planning architectures. We additionally evaluate three published architectures that approach planning. **SayPlan** grounds its plan in the scene graph the agent has accumulated so far and re-plans on detected mismatch. **LookPlanGraph** augments the agent's memory graph during execution and corrects actions in flight when the world deviates from the plan. **AutoGPT+P** delegates symbolic planning to a PDDL solver invoked as a tool.

5.2 Lifelong rollouts (ReAct baseline)

We run the ReAct baseline across two reference houses (the 5-room *simple* house and the 11-room *big* house) for the three models that are currently enabled in the lifelong experiment configuration. Per-scenario scores are visualised as a polar (rose) plot in Figure 4, with one polygon per house overlaid on each model's panel; Table 2 reports the four headline metrics from Section 4.4 aggregated over all scenarios. Cells marked "—" denote model/house combinations for which runs have not yet been collected.

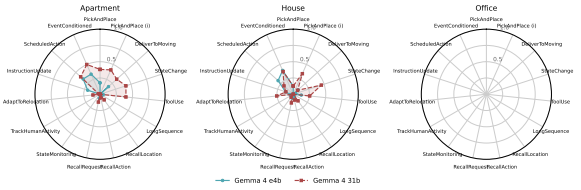

Figure 4: Per-scenario score for the ReAct baseline across 15 CapabilityBench scenarios, one panel per model. Each polygon corresponds to one of the two reference houses in LivingGrid.

Model	SR (%)	Score	Exec. (%)	USD /run
Simple house				
Gemma e4b	—	—	—	—
Gemma 31b	13.3	0.20	100.0	0.039
Gemini 3.5	—	—	—	—
Big house				
Gemma e4b	—	—	—	—
Gemma 31b	12.2	0.17	100.0	0.056
Gemini 3.5	—	—	—	—

Table 2: ReAct baseline...

5.3 RQ1: model scaling within a fixed ReAct loop

Holding the ReAct loop fixed, how does the choice of language model affect the capability profile?

5.4 RQ2: planning architecture vs. ReAct on a fixed model

Holding the underlying language model fixed, we compare ReAct to three planning architectures on each capability cluster.

5.5 RQ3: memory recall vs. simulated-time gap

The Memory scenarios are parameterised by the simulated time gap inserted between the observation episode and the recall episode. We report per-scenario score as a function of this gap to characterise how recall degrades when the intermediate human life is longer.

5.6 RQ4: failure-mode analysis

A qualitative pass over rollouts categorises typical failures: hallucinated affordances, forgotten goal, action loops, ignored mid-execution updates, premature termination, navigation deadlock.

5.7 Auxiliary results

We additionally report executability per model per cluster and token cost per scenario per model. Executability gives a grounding-quality signal independent of task success, and the cost table makes model comparisons explicit.

Limitations

CapabilityBench is written in the language of household robotics, but its agent contract remains a discrete grid, a textual scene description, and structured JSON observations and actions. This abstraction is deliberate: it isolates the cognitive layer of embodied agency—instruction grounding, memory across gaps, temporal reasoning, and online adaptation—from pixel-level perception, low-level control, and physical manipulation, while keeping experiments reproducible without a shared robotic platform. Comparative results should therefore be read as cognitive-layer comparisons: relative model performance, relative planning-architecture performance, recall curves over inter-episode gaps, and failure-mode structure.

The abstraction also limits calibration. Absolute success rates, time efficiency, and executability are upper bounds rather than deployment predictions, because perception is noiseless within open containment chains, observed entities are fully described, action outcomes are deterministic, and grid-aligned navigation hides failures that a physical robot would expose. Future work should connect CapabilityBench to photorealistic simulators and hardware, sample layouts from real floorplan datasets, expand the object and routine catalogues, and add robot speech actions together with communication-focused scenarios.

The scenario pool in this release covers five capability clusters but does not yet probe spatial reasoning, instruction ambiguity, or scenarios that combine clusters within a single episode. Human agents follow scripted intents rather than learned policies, so the diversity of background behaviour is bounded by the routine and persona catalogue we author. Finally, the language-model evaluation uses a fixed temperature and a fixed prompt template per architecture; sensitivity to prompt phrasing and to decoding hyperparameters is not characterised here.

Ethical Considerations

CapabilityBench evaluates language-model agents in a synthetic household simulator. All scenes, items, and human agents are procedurally generated from configuration files we author; no personal data, real floor plans, or recordings of real people

are used, and no human subjects participate in evaluation. The benchmark therefore raises no direct data-privacy or human-subjects concerns.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, and 25 others. 2022. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning*.

Tobias Birr and 1 others. 2024. Autogpt+p: Affordance-based task planning with large language models. *arXiv preprint arXiv:2402.10778*.

Matthew Chang and 1 others. 2024. Partnr: A benchmark for planning and reasoning in embodied multi-agent tasks. *arXiv preprint arXiv:2411.00081*.

Jae-Woo Choi, Youngwoo Yoon, Hyobin Ong, Jaehong Kim, and Minsu Jang. 2024. Lota-bench: Benchmarking language-oriented task planners for embodied agents. *arXiv preprint arXiv:2402.08178*.

Matt Deitke, Eli VanderBilt, Alvaro Herrasti, Luca Weihs, Kiana Ehsani, Jordi Salvador, Winson Han, Eric Kolve, Aniruddha Kembhavi, and Roozbeh Mottaghi. 2022. Procthor: Large-scale embodied ai using procedural generation. In *Advances in Neural Information Processing Systems*.

Miguel Gonzalez-Pumariega and 1 others. 2025. Robotouille: An asynchronous planning benchmark for llm agents. *arXiv preprint arXiv:2502.05227*.

Fan Hu and 1 others. 2025. 3dllm-mem: Long-term spatial-temporal memory for embodied 3d large language model. *arXiv preprint arXiv:2505.22657*.

Yingdong Hu, Fanqi Lin, Tong Zhang, Li Yi, and Yang Gao. 2023. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*.

Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, and 1 others. 2022. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*.

Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, Aniruddha Kembhavi, Abhinav Gupta, and Ali Farhadi. 2017. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv:1712.05474*.

Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same task, more tokens: the impact of input length on the reasoning performance of large language models. *arXiv preprint arXiv:2402.14848*.

Chengshu Li, Fei Xia, Roberto Martín-Martín, Michael Lingelbach, Sanjana Srivastava, Cem Gokmen, Shyamal Buch, Kent Vainio, Karen Liu, Hyowon Gweon, Jiajun Wu, and Li Fei-Fei. 2024a. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*.

Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, and 1 others. 2024b. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*.

Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. Llm+p: Empowering large language models with optimal planning proficiency. *Preprint*, arXiv:2304.11477.

Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. Nolima: Long-context evaluation beyond literal matching. *arXiv preprint arXiv:2502.05167*.

Nikita Onishchenko and 1 others. 2025. Lookplangraph: Embodied instruction following method with vlm graph augmentation. *arXiv preprint arXiv:2512.21243*.

Aishwarya Padmakumar, Jesse Thomason, Ayush Shrivastava, Patrick Lange, Anjali Narayan-Chen, Spandana Gella, Robinson Piramuthu, Gokhan Tur, and Dilek Hakkani-Tur. 2022. Teach: Task-driven embodied agents that chat. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. 2018. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8494–8502.

Xavier Puig, Tianmin Shu, Shuang Li, Zilin Wang, Yuan-Hong Liao, Joshua B. Tenenbaum, Sanja Fidler, and Antonio Torralba. 2021. Watch-and-help: A challenge for social perception and human-ai collaboration. In *International Conference on Learning Representations*.

Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian D Reid, and Niko Suenderhauf. 2023. Sayplan: Grounding large language models using 3d scene graphs for scalable task planning. *CoRR*.

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke

Zettlemoyer, and Dieter Fox. 2020a. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020b. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*.

Sanjana Srivastava, Chengshu Li, Michael Lingelbach, Roberto Martín-Martín, Fei Xia, Kent Vainio, Zheng Lian, Cem Gokmen, Shyamal Buch, Karen Liu, Silvio Savarese, Hyowon Gweon, Jiajun Wu, and Li Fei-Fei. 2021. Behavior: Benchmark for everyday household activities in virtual, interactive, and ecological environments. *arXiv preprint arXiv:2108.03332*.

Ying Su, Zhan Ling, Haochen Shi, Jiayang Cheng, Yauwai Yim, and Yangqiu Song. 2024. Actplan-1k: Benchmarking the procedural planning ability of visual language models in household activities. *arXiv preprint arXiv:2410.03907*.

Andrew Szot, Alexander Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Chaplot, Oleksandr Maksymets, Aaron Gokaslan, Vladimir Vondrus, Sameer Dharur, Franziska Meier, Wojciech Galuba, Angel Chang, Zsolt Kira, Vladlen Koltun, Jitendra Malik, and 2 others. 2021. Habitat 2.0: Training home assistants to rearrange their habitat. In *Advances in Neural Information Processing Systems*.

Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. Scienceworld: Is your agent smarter than a 5th grader? *arXiv preprint arXiv:2203.07540*.

Zixuan Wang, Bo Yu, Junzhe Zhao, Wenhao Sun, Sai Hou, Shuai Liang, Xing Hu, Yinhe Han, and Yiming Gan. 2024. Karma: Augmenting embodied ai agents with long-and-short term memory systems. *arXiv preprint arXiv:2409.14908*.

Karmesh Yadav and 1 others. 2025. Findingdory: A benchmark to evaluate memory in embodied agents. *arXiv preprint arXiv:2506.15635*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Yichi Zhang and 1 others. 2024. Et-plan-bench: Embodied task-level planning benchmark towards spatial-temporal cognition with foundation models. *arXiv preprint arXiv:2410.14682*.

Zihan Zhou and 1 others. 2024. Hazard challenge: Embodied decision making in dynamically changing environments. *arXiv preprint arXiv:2401.12975*.

Yifeng Zhu and 1 others. 2026. When robots do the chores: A benchmark and agent for long-horizon household task execution. *arXiv preprint arXiv:2605.14504*.

A Observation snapshot

[Appendix A: full JSON schema of the observation snapshot, formalising the fields summarised in Section 4.3. To be added in the camera-ready version.]

B Action signatures

[Appendix B: per-verb signatures (`move_forward`, `turn_left`, `turn_right`, `go_to`, `pickup`, `place`, `open`, `close`, `turn_on`, `turn_off`, `give`, `cut`, `pour`, `wash`, `say`, `wait`, `sleep_until_woken`) with arguments, return semantics, and the conditions under which they fail.]

C Configuration examples

LivingGrid scenes are assembled from small YAML configuration files. The examples below show representative fragments for a house template, a room template, and the object catalog.

House template.

```
grid:
  rows: 70
  cols: 70
rooms:
  - name: kitchen
    relative_size: 0.13
    width_to_height_ratio: 1.0
  - name: living-room
    relative_size: 0.15
    width_to_height_ratio: 2.0
  - name: bedroom
    relative_size: 0.15
    width_to_height_ratio: 1.0
connections:
  - rooms: [kitchen, living-room]
    no_wall: false
  - rooms: [living-room, bedroom]
    no_wall: false
```

Room template.

```
room_type: kitchen
assets_required:
  - asset_type: table
```

```
    count: 1
  - asset_type: sink
    count: 1
  - asset_type: chair
    count: 4
  - asset_type: fridge
    count: 1
  - asset_type: countertop
    count: 2
assets_optional: []
```

Catalog fragments.

```
surfaces:
  drawer:
    openable: true
    interaction: inside
  shelf:
    interaction: on_top
  top:
    interaction: on_top

items:
  apple:
    properties:
      default: [red, cuttable]
  box:
    properties:
      default: [tan, openable]
    surfaces:
      - id: insides
        interaction: in
  dish-soap:
    properties:
      default: [green, pourable]

assets:
  cabinet:
    properties:
      default: [brown, openable]
    width: 3
    height: 2
    passable: false
    surfaces:
      - properties: [top]
        type: shelf
        requires_open_parent: true
        allowed_objects:
          max: 6
         items: [mug, cup, keys, glasses]
      - properties: [bottom]
        type: shelf
        requires_open_parent: true
        allowed_objects:
          max: 6
          items: [plate, keys, glasses]
```

D Capability taxonomy and milestones

This appendix collects the capability taxonomy and per-scenario milestone definitions referenced in Section 4.2. Implemented scenarios use the milestones present in the current release. Specified scenarios fix the milestones below; their predicates and weights are part of the benchmark specification.

Temporal: scheduled action. Latches `not_acted_too_early` (weight 0.3, latches at the first decision tick after the target time if the state is still untouched), `acted_in_window` (0.5, latches if the action fires within ± 2 ticks of the target time), and `noop_window_clean` (0.2, latches if no other manipulation occurs between the instruction and the execution).

Temporal: event-conditioned action. Latches `not_acted_before_event` (0.2), `latency_under_K` (0.5, latches if the response action fires within K ticks of the trigger event), and `pizza_intact` (0.3).

Temporal: short-horizon ordering. Latches `closed_window` (0.4) and `turned_on_lamp_after_close` (0.6).

Long-horizon: set the table. Latches a per-seat completion milestone (0.20 each, four seats) and `arrangement_consistent` (0.20).

Long-horizon: tidy after party. Latches a per-class completion milestone (0.20 each, four classes) and `completeness` (0.20).

Long-horizon: multi-room laundry. Latches a per-room completion milestone (0.25 each, three bedrooms) and `all_in_basket` (0.25).

Each Temporal and Long-horizon scenario also carries a `success_threshold_ticks` cap so the scheduler can move on if the agent stalls.

E Hyperparameters and prompt templates

[Appendix D: prompt templates for the ReAct primary baseline and each planning-architecture baseline, plus generation hyperparameters per model.]

Capability	Description	Example
Foundational		
PickAndPlace	Move an object to a target surface or container, including open-container variants.	"put the mug on the shelf"
Deliver	Bring an object to a human target that may move after issuing the request.	"bring me the book"
StateChange	Change the state of an object or asset without moving it.	"turn on the lamp"
ToolUse	Acquire and apply an intermediate tool before acting on the target.	"wash the plate"
Memory		
Observed location	Recall a location observed in an earlier episode.	"bring me my glasses"
Own action trace	Recall where the robot previously found or moved an object.	"return it to where you found it"
Past instruction	Resolve a later request that refers to an earlier dialogue turn.	"bring the item I asked for this morning"
Dynamic		
Adversarial unpacking	Adapt while a human keeps changing the scene during execution.	"keep the bed clear while I unpack"
Temporal		
Scheduled action	Execute an action at a specified simulated time.	"set the oven to bake mode at 18:00"
Event-conditioned action	Wait for an event before acting.	"when the oven beeps, take the pizza out"
Short-horizon ordering	Preserve ordering constraints over a short action sequence.	"first close the window, then turn on the lamp"
Long-horizon		
Set the table	Decompose a structured task into repeated per-place settings.	"set the table for four"
Tidy after party	Route heterogeneous object classes to different destinations.	"clean up after the party"
Multi-room laundry	Search across rooms and collect a distributed object class.	"collect dirty laundry from all bedrooms"

Table 3: Capability taxonomy in CapabilityBench.